# Неделя 2. Контекст и feedback loop

Минимальный контекст-пакет и цикл проверок, замкнутый на командах проекта

# План лекции

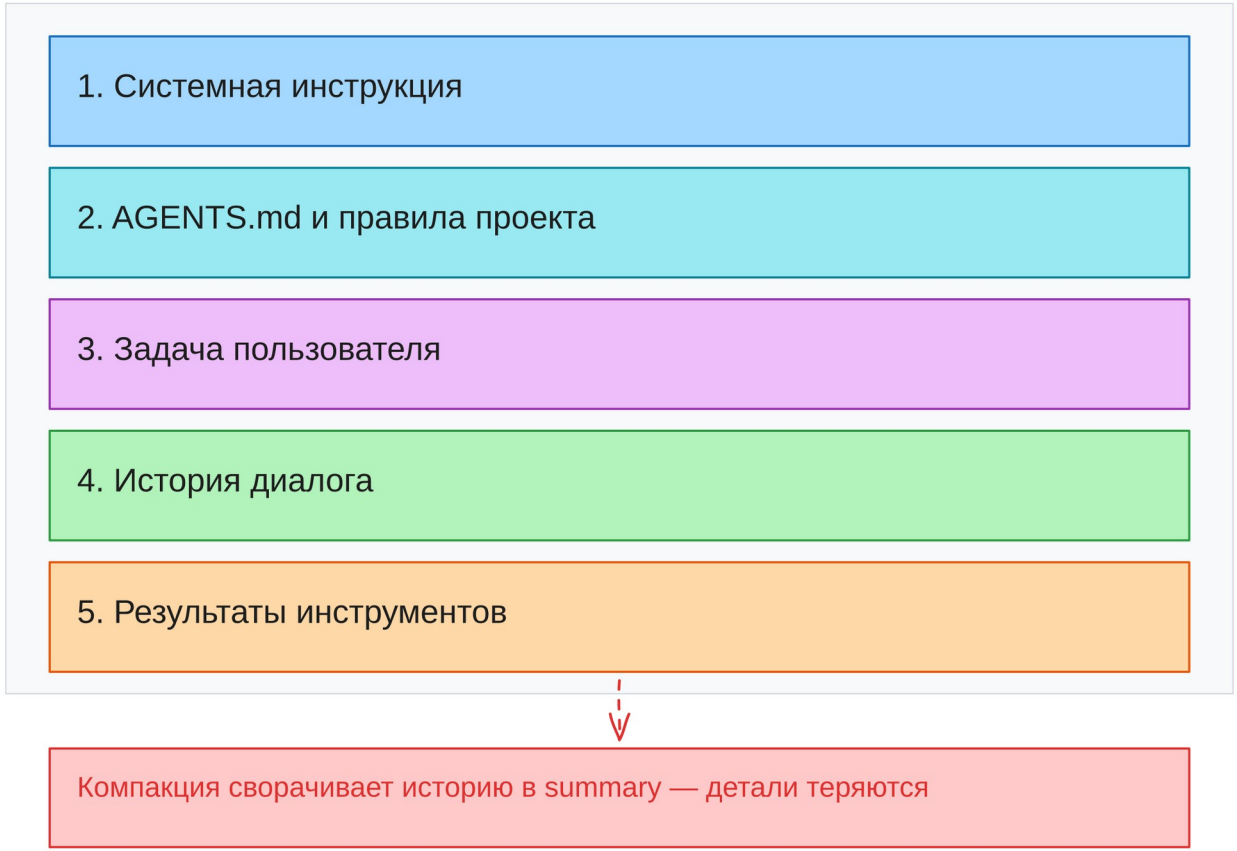
- Релевантность контекста важнее объёма
- AGENTS.md и знания проекта
- Состояние между сессиями
- Feedback loop и порядок проверок
- Как читать вывод команд
- Типичные ошибки и разобранный пример

# Контекст — минимум, без которого задача не решается

- Постановка из недели 1
- 3–7 файлов, которые участвуют в изменении
- Спека или user story
- Ограничения проекта
- Команды проверки
- Список неизвестностей

# Пять слоёв контекста

## Что модель видит в одном вызове




контекст одного вызова

Чем ниже слой, тем быстрее он устаревает

# «Прочитать весь репозиторий» — плохая стратегия

- Окно контекста конечно, а полный проект дороже
- Внимание размывается: похожие файлы путают
- Мёртвый код и старые спеки агент читает как руководство

# Сначала карта и указатели, потом правки

- Карта: где backend, frontend, спеки, тесты
- Указатели: логика — файл, тест — файл, спека — путь
- Неверная карта — потеря времени, неверный указатель — не тот файл

# Полноту контекст-пакета проверяют до правок

- Попросить перечислить файлы к изменению
- Попросить команды, которыми будет проверять
- Список расходится — контекст неполный или постановка размыта
- Пять минут сверки экономят час переделки

# AGENTS.md — инструкция агенту, а не документация

- Документация объясняет, как устроен проект и почему
- AGENTS.md говорит, что делать и чего не делать
- Команды проверки, политика веток, границы согласования
- Короткая карта репозитория с указателями на подробности

# Команды из AGENTS.md копируются в терминал дословно

- Команды дословно, без пересказа
- Порядок: ветка до правки, проверки до merge, merge до push
- Запреты: миграции, .env, docker-compose.yml, workflow
- Карта: backend, frontend, тесты, спеки, дизайн-система
- Без пересказа архитектуры на десять экранов

# Пробел в AGENTS.md фиксируют, но правку согласуют

- Дважды объяснили одну команду — пробел в инструкции
- Агент полез в прод-каталог — граница не записана
- На этой неделе: формулировка и обоснование в results.md
- Правку AGENTS.md принимает автор проекта

# Сессия не помнит предыдущую — помнят файлы

- История сессии живёт только до её конца
- Постоянное: AGENTS.md, спека, план, results.md
- results.md: что сделано, каким способом, что вышло, что осталось

# Компакция сохраняет смысл, но теряет детали

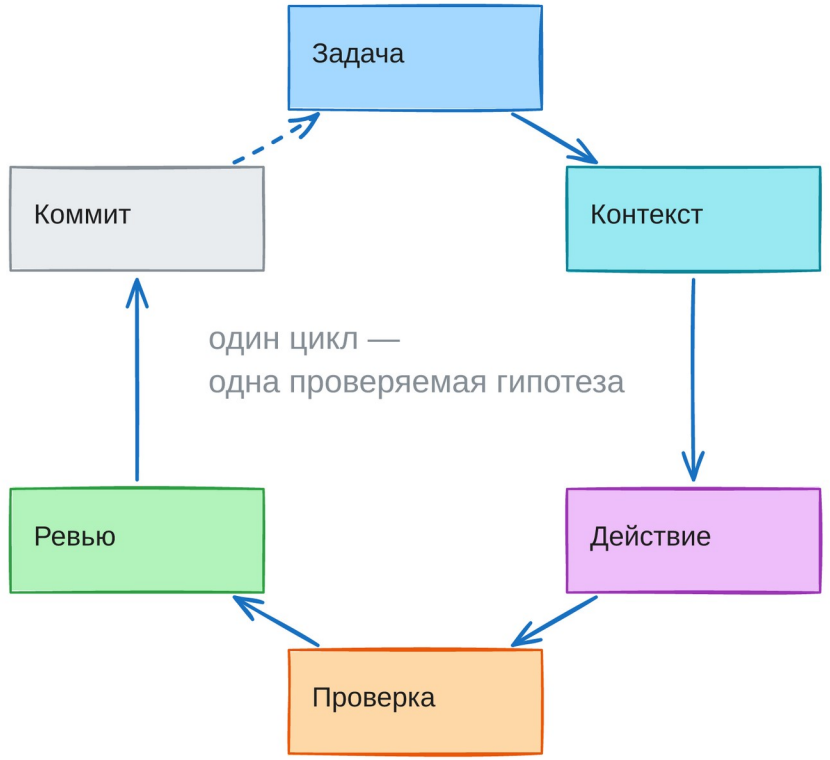
- Harness сворачивает старые сообщения в summary
- Теряются имена полей, номера строк, критерии, ссылки
- Решения записывать сразу, а не надеяться на память
- На неделе 4 — .memory-bank/: index, product, architecture, decisions

# Feedback loop замыкается на командах, а не на заверениях

- Цикл: сделал → проверил → прочитал → исправил
- Без цикла агент не знает, компилируется ли код
- Человек проверяет не «написал ли», а «какой командой подтверждено»
- Команда не запускалась — результата нет, есть только текст

# Цикл обратной связи

## Цикл обратной связи




Короткий цикл → быстрее обратная связь → меньше потерь контекста

# Проверки идут от дешёвых к дорогим

build → формат → таргетные тесты → frontend → гейты → E2E

```
dotnet build CollegeLMS.slnx
dotnet csharpier check .
dotnet test CollegeLMS.slnx --filter FullyQualifiedName~{Name}
(cd CollegeLMS.Next && npm run build)
(cd CollegeLMS.Next && npm run check:design-system)
(cd CollegeLMS.Next && npx playwright test {spec})
```

# Playwright допустим только на изолированном стенде

- Worktree изолирует файлы, а не сервер и базу
- localhost:3000 и reuseExistingServer ведут на общий стенд
- Нужны свой PLAYWRIGHT_BASE_URL и одноразовая база
- Иначе в журнале: «не запускался: нет изолированного стенда»

# Код возврата — не истина

- Ноль — успех, ненулевой — ошибка, но этого мало
- Первая ошибка — причина, остальные — следствия
- Полный лог в файл, а не пересказ хвоста
- Ложное зелёное: код 0, а проверено не то
- Фильтр без совпадений, кэш, пропуск, не тот спек, скрытая ошибка
- В журнал — что именно проверялось

# Пять ошибок, которые обесценивают проверку

- Проверять не то: frontend build при правке backend
- Игнорировать формат: csharpier падает, а правят логику
- Чинить тест вместо кода — потеря проверки
- Гонять E2E на каждую правку текста
- Работать в прод-каталоге: там контейнеры и пользователи

# Зелёный build: код 0 и ни одной ошибки

Ожидаемый вывод; сборка в этой итерации не запускалась

```
$ dotnet build CollegeLMS.slnx
...
Build succeeded.
    0 Warning(s)
    0 Error(s)
```

# Зелёный прогон — это выбранные и выполненные тесты

В журнал: cwd, фактическая команда, число тестов, код возврата, первая ошибка, путь к логу

```
$ dotnet test CollegeLMS.slnx --filter FullyQualifiedName~StudentFilterTests
Passed!  - Failed:     0, Passed:     7, Skipped:     0, Total:     7
```

# Резюме: что запомнить

- Контекст — минимум релевантного, а не весь репозиторий
- Сначала карта и указатели, потом правки
- AGENTS.md — команды, порядок, запреты, карта
- Память — это файлы, а не история чата
- Цикл замыкается на команде, а не на слове «готово»
- Зелёный результат подтверждается логом и числом тестов

# Вопросы для самопроверки

- Почему «прочитай весь репозиторий» ухудшает результат?
- Чем AGENTS.md отличается от документации проекта?
- Что теряется при компакции и как это компенсировать?
- Почему проверки идут от дешёвых к дорогим?
- Какой прогон считается зелёным, а какой — ложным зелёным?
- Как отличить ошибку кода от ошибки проверки?